Московский Физико–Технический Институт

Численные методы
Задание №9

ФИО: Лазарь Владислав Игоревич
Группа: Б01-207а
Почта: lazar.v@phystech.edu

Долгопрудный, 2026 г.

Введение

Задание посвящено оптимизации операции умножения матриц $C = A \times B$ для современных процессорных архитектур. Основное внимание уделяется эффективному использованию иерархии памяти (`L1` кэш) и векторных инструкций (`SIMD`). В данной работе рассматриваются матрицы с размерами, кратными степеням двойки, что позволяет избежать накладных расходов на обработку краевых условий при векторизации.

В данной работе процесс оптимизации рассматривается итеративно. Реализация разделена на следующие этапы:

1. **Наивный алгоритм** — обычная реализация умножения строк матрицы A на столбцы матрицы B.

2. **Оптимизация доступа к памяти** — транспонирование матрицы B для обеспечения линейного прохода по памяти.

3. **Векторизация с использованием интринсиков AVX2** — прямое управление 256-битными регистрами процессора.

   Использование инструкций `_mm256_loadu_ps` и `_mm256_fmadd_ps` позволит обрабатывать 8 `float` элементов за одну итерацию и использовать блоки FMA.

4. **Регистровое блокирование** — использование векторных регистров в качестве аккумуляторов. В этой реализации подразумевается удержание промежуточных сумм в регистрах процессора на протяжении всего внутреннего цикла. Так мы сможем минимизировать количество инструкций `LOAD/STORE` в кэш или память.

1. Наивное умножение матриц

Классическое определение произведения матриц $A \in \mathbb{R}^{M \times K}$ и $B \in \mathbb{R}^{K \times N}$ строится на вычислении скалярного произведения строк левой матрицы на столбцы правой. Результирующий элемент $C_{i,j}$ определяется следующим образом:

$$C_{ij} = \sum_{k=1}^{K} A_{ik} \cdot B_{kj},$$

где $i = \overline{1, M}$ — индекс строки матрицы A; $j = \overline{1, N}$ — индекс столбца матрицы B; k — индекс суммирования.

Или в матричном виде:

$$\underbrace{\begin{pmatrix} a_{11} & \cdots & a_{1k} & \cdots & a_{1K} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ \boxed{a_{i1} \quad \cdots \quad a_{ik} \quad \cdots \quad a_{iK}} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ a_{M1} & \cdots & a_{Mk} & \cdots & a_{MK} \end{pmatrix}}_{\text{матрица } A \in \mathbb{R}^{M \times K}} \times \underbrace{\begin{pmatrix} b_{11} & \cdots & \boxed{b_{1j}} & \cdots & b_{1N} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ b_{k1} & \cdots & b_{kj} & \cdots & b_{kN} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ b_{K1} & \cdots & b_{Kj} & \cdots & b_{KN} \end{pmatrix}}_{\text{матрица } B \in \mathbb{R}^{K \times N}} = \underbrace{\begin{pmatrix} c_{11} & \cdots & c_{1j} & \cdots & c_{1N} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ c_{i1} & \cdots & \boxed{c_{ij}} & \cdots & c_{iN} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ c_{M1} & \cdots & c_{Mj} & \cdots & c_{MN} \end{pmatrix}}_{\text{матрица } C \in \mathbb{R}^{M \times N}}$$

Программная реализация наивного умножения

Логику реализации можно понять из иллюстрации матричного умножения выше.

```
// int M, K, N = ...
void matmul_naive(const float *A, const float *B, float *C) {
  for (int i = 0; i < M; ++i) {      // Passing through the lines of A
    for (int j = 0; j < N; ++j) {    // Passing through the columns of B
      float part_sum = 0.0f;

      for (int k = 0; k < K; ++k) { // Scalar product
        part_sum += A[i * K + k] * B[k * N + j];
      }

      C[i * N + j] = part_sum;
    }
  }
}
```

Эффективно ли?

С точки зрения архитектуры процессора, данный алгоритм имеет большой недостаток — плохую локальность данных по столбцам. Во внутреннем цикле (по k) происходит обращение к элементам B_{kj}. При каждом шаге k мы переходим на новую строку матрицы B. Поскольку матрицы хранятся в памяти по строкам (`Row-major order`), каждый такой переход заставляет процессор прыгать через N элементов. Это приводит к высокой вероятности промаха кэша, невозможности эффективного предсказания данных.

2. Умножение матриц с оптимизацией доступа к памяти

Исходя из описанных проблем, перейдем к банальнейшему улучшению. **Математически** просто транспонируем матрицу B, тогда с точки зрения локальности данных все будет отлично, ведь теперь прежние столбцы матрицы B станут строками, и процессор не будет каждый раз прыгать через N элементов — данные теперь лежат линейно.

$$\underbrace{\begin{pmatrix} a_{11} & \cdots & a_{1k} & \cdots & a_{1K} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ \boxed{a_{i1} \quad \cdots \quad a_{ik} \quad \cdots \quad a_{iK}} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ a_{M1} & \cdots & a_{Mk} & \cdots & a_{MK} \end{pmatrix}}_{\text{матрица } A \in \mathbb{R}^{M \times K}} \times \underbrace{\begin{pmatrix} b_{11} & \cdots & b_{k1} & \cdots & b_{K1} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ \boxed{b_{1j} \quad \cdots \quad b_{kj} \quad \cdots \quad b_{Kj}} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ b_{1N} & \cdots & b_{kN} & \cdots & b_{KN} \end{pmatrix}}_{\text{матрица } B^T \in \mathbb{R}^{N \times K}} = \underbrace{\begin{pmatrix} c_{11} & \cdots & c_{1j} & \cdots & c_{1N} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ c_{i1} & \cdots & \boxed{c_{ij}} & \cdots & c_{iN} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ c_{M1} & \cdots & c_{Mj} & \cdots & c_{MN} \end{pmatrix}}_{\text{матрица } C \in \mathbb{R}^{M \times N}}$$

Однако, **программно** это не очень умно, ведь нам каждый раз придется транспонировать матрицу B. Это трата ресурсов на пустом месте. Рассмотрим на простейшем примере умножения матриц $A \in \mathbb{R}^{2\times 2}$ и $B \in \mathbb{R}^{2\times 2}$ как можно избежать транспонирования и при этом воспользоваться свойством пространственной локальности данных:

$$\begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \times \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix} = \begin{pmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{pmatrix}$$

Здесь имеем:

$$c_{11} = a_{11}b_{11} + a_{12}b_{21}$$

$$c_{12} = a_{11}b_{12} + a_{12}b_{22}$$

Мы можем не сразу считать конкретный элемент c_{ij}, а частично накапливать его сумму:

- Будем фиксировать строку i матрицы A. В этой строке фиксируем элемент $a_{ik} \equiv a$.
- Далее будем пробегаться по строке j матрицы B. Тогда можно утверждать, что

$$c_{ij} = a \cdot b_{kj}$$

Тем самым, мы проходимся в матрице B по элементам построчно (по j; фиксация k была пунктов выше вместе с фиксацией элемента a_{ik}), а значит воспользовались пространственной локальностью. А также мы нигде не транспонировали матрицы. То есть воспользовались исключительно перестановкой циклов в коде.

Программная реализация оптимизированного умножения

```
// int M, K, N = ...
void matmul_optimized(const float *A, const float * B, float * C) {
  for (int i = 0; i < M; ++i) {
    for (int k = 0; k < K; ++k) {
      float fixed_a = A[i * K + k];

      for (int j = 0; j < N; ++j)
        C[i * N + j] += fixed_a * B[k * N + j];
    }
  }
}
```

3. Векторизация с использованием инструкций AVX2 и FMA

Предыдущая оптимизация позволила эффективно использовать кэш-память, обеспечив линейный доступ к данным. Однако на уровне вычислительного ядра мы все еще работаем в скалярном режиме. Одна инструкция обрабатывает одну пару чисел. Процессоры архитектуры `x86_64` поддерживают технологию `SIMD`, позволяющую выполнять одну операцию над целым вектором данных.

Для реализации этого подхода воспользуемся расширением `AVX2`. Основным инструментом здесь выступают 256-битные регистры (`YMM`), каждый из которых способен вместить 8 чисел типа `float`.

Вместо того чтобы в самом внутреннем цикле умножать `fixed_a` на один элемент B, мы будем:

1. Загружать `fixed_a` и размножать его во все 8 ячеек векторного регистра.
2. Далее загружать сразу 8 последовательных элементов матрицы B в другой регистр.
3. В итоге перемножать эти векторы и прибавлять результат к 8 элементам матрицы C за одну инструкцию.

`FMA` позволяет вычислить выражение `a·b+c` за один такт процессора без промежуточного округления.

Программная реализация с интринсиками

```
// int M, K, N = ...
void matmul_avx2(const float *A, const float *B, float *C) {
  for (int i = 0; i < M; ++i) {
    for (int k = 0; k < K; ++k) {
      // Load element A[i][k] and propagate it by the entire register
      __m256 vec_a = _mm256_set1_ps(A[i * K + k]);

      // Go through the row of the matrix B and C
      for (int j = 0; j < N; j += 8) {
        // Load 8 elements from B[k][j], ..., B[k][j+7]
        __m256 vec_b = _mm256_loadu_ps(&B[k * N + j]);

        // Load 8 elements from C[i][j], ..., C[i][j+7]
        __m256 vec_c = _mm256_loadu_ps(&C[i * N + j]);

        // C = A * B + C
        vec_c = _mm256_fmadd_ps(vec_a, vec_b, vec_c);

        // Saving the result back to the C
        _mm256_storeu_ps(&C[i * N + j], vec_c);
      }
    }
  }
}
```

4. Регистровое блокирование

В предыдущей реализациия мы векторизовали вычисления, но продолжали обращаться к памяти для записи результата матрицы C на каждой итерации по k. В данной реализации был применен метод регистрового блокирования.

Выделим сразу несколько векторных регистров (в данном случае 4 регистра `YMM`) для хранения промежуточных сумм блока элементов 2×16. Перед началом внутреннего цикла по k регистры обнуляются. Вся арифметика накопления происходит внутри процессора без обращения к кэшу или `RAM`. Загружая один вектор из матрицы B, мы используем его сразу для умножения на несколько разных элементов из матрицы A (находящихся в разных строках).

Программная реализация регистрового блокирования

```
// int M, K, N = ...
void matmul_multi_reg(const float *A, const float *B, float *C) {
  // Go through the rows of matrix A with step 2
  for (int i = 0; i < M; i += 2) {
    // Go through the cols of matrix B with step 16 (2 registers of 8
       floats)
    for (int j = 0; j < N; j += 16) {

      // 4 accumulators for the 2x16 block
      __m256 acc00 = _mm256_setzero_ps(); // Row i,   cols j  , ..., j+7
      __m256 acc01 = _mm256_setzero_ps(); // Row i,   cols j+8, ..., j+15
      __m256 acc10 = _mm256_setzero_ps(); // Row i+1, cols j  , ..., j+7
      __m256 acc11 = _mm256_setzero_ps(); // Row i+1, cols j+8, ..., j+15

      for (int k = 0; k < K; ++k) {
        // Load elements from A
        __m256 a0 = _mm256_set1_ps(A[i * K + k]);
        __m256 a1 = _mm256_set1_ps(A[(i + 1) * K + k]);

        // Load 16 elements from B
        __m256 b0 = _mm256_loadu_ps(&B[k * N + j]);
        __m256 b1 = _mm256_loadu_ps(&B[k * N + j + 8]);

        // Multiply and accumulate the sum
        acc00 = _mm256_fmadd_ps(a0, b0, acc00);
        acc01 = _mm256_fmadd_ps(a0, b1, acc01);
        acc10 = _mm256_fmadd_ps(a1, b0, acc10);
        acc11 = _mm256_fmadd_ps(a1, b1, acc11);
      }

      // Upload 4 registers into memory once
      _mm256_storeu_ps(&C[i * N + j], acc00);
      _mm256_storeu_ps(&C[i * N + j + 8], acc01);
      _mm256_storeu_ps(&C[(i + 1) * N + j], acc10);
      _mm256_storeu_ps(&C[(i + 1) * N + j + 8], acc11);
    }
  }
}
```

Методология измерений

- **Язык и компилятор**: `C++17`, компилятор `g++` с флагами оптимизации `-O2 -mavx2 -mfma -ffast-math`. Использование `-O2` вместо `-O3` выбрано осознанно, чтобы исключить автоматическую векторизацию и наглядно сравнить ручную оптимизацию с базовой логикой. Автовектория от компилятора будет показано далее.
- **Метрика измерения**: использование регистра `TSC` через инструкцию `__rdtsc()`.
- **Контроль точности**: каждый результат сравнивается с эталонным значением `Naive MatMul` с допустимой погрешностью $\varepsilon = 10^{-3}$.

Эксперименты запускались на платформе с архитектурой `AMD Zen 3`:

- Процессор: `AMD Ryzen 7 5800U` (8 ядер, 16 потоков)
- Микроархитектура: `Zen 3 (Cezanne)`.
- Операционная система: `Linux Ubuntu`.
- Тактовая частота: базовая 1.9 ГГц.
- Наборы инструкций: `AVX`, `AVX2`, `FMA3`.

Иерархия кэш-памяти:

- `L1d`: 32 КБ на ядро (всего 256 КБ). Хватает на матрицу 64×64.
- `L2`: 512 КБ на ядро (всего 4 МБ). Хватает на матрицу 256×256.
- `L3`: 16 МБ (общий на все ядра). Хватает на матрицу 1024×1024.

Результаты на -O2

Размер	Метод	Такты	Ускорение
32x32	MatMul naive	24795	1.00
	MatMul opt	33934	0.73
	AVX2 + FMA	5453	4.55
	MatMul reg	2166	11.45
64	MatMul naive	239248	1.00
	MatMul opt	294918	0.81
	AVX2 + FMA	37962	6.30
	MatMul reg	19019	12.58
128	MatMul naive	5696295	1.00
	MatMul opt	5320475	1.07
	AVX2 + FMA	333982	17.06
	MatMul reg	196707	28.96
256	MatMul naive	44472293	1.00
	MatMul opt	17338564	2.56
	AVX2 + FMA	1584695	28.06
	MatMul reg	1199242	37.08
512	MatMul naive	530395469	1.00
	MatMul opt	127128335	4.17
	AVX2 + FMA	14429417	36.76
	MatMul reg	12985512	40.85
1024	MatMul naive	7376729822	1.00
	MatMul opt	998968529	7.38
	AVX2 + FMA	108708880	67.86
	MatMul reg	184031511	40.08

`g++ -O2 -mavx2 -mfma -ffast-math`

Анализ результатов на -O2

1. На малых размерах (32, 64) обычная перестановка циклов не дает преимуществ и даже замедляет выполнение (ускорение <1.0). Это связано с тем, что накладные расходы на управление индексами превышают пользу, так как матрицы полностью помещаются в `L1` кэш. Однако на размере 1024 метод показывает ускорение 7.38x, подтверждая важность линейного обхода памяти при выходе за пределы кэша.

2. Векторизация (`AVX2` и `FMA`) демонстрирует стабильный рост эффективности. На размере 1024 достигнут пик в 67.86x.

3. Регистровое блокирование с использованием 4-х аккумуляторов оказалось наиболее эффективным на средних дистанциях. На размере 512 он показал лучший результат (40.85x),

обходя стандартный `AVX2`. Это подтверждает, что уменьшение количества обращений к портам памяти (`Load`/`Store`) за счет переиспользования регистров дает значительный прирост.

4. При размере матриц 1024 `AVX2` обошел `MatMul reg` почти в 1.7 раза. Метод `MatMul reg` обрабатывает две строки за раз. На большой матрице 1024×1024 это приводит к слишком быстрому вытеснению данных из кэша `L1` (32 КБ на ядро). `AVX2`, работающий более линейно, в данном случае создает меньше конфликтов в кэш-памяти.

   Таким образом, для матриц размера 1024 и более требуется внедрение блочного алгоритма (`Cache Tiling`), так как текущие оптимизации упираются в пропускную способность памяти, а не в вычислительную мощность ядра.

Результаты на `-O3`

Размер	Метод	Такты	Ускорение
32x32	MatMul naive	12369	1.00
	MatMul opt	4123	3.00
	AVX2 + FMA	3800	3.25
	MatMul reg	1653	7.48
64	MatMul naive	129314	1.00
	MatMul opt	28937	4.47
	AVX2 + FMA	32870	3.93
	MatMul reg	17784	7.27
128	MatMul naive	5598065	1.00
	MatMul opt	163381	34.26
	AVX2 + FMA	191577	29.22
	MatMul reg	118446	47.26
256	MatMul naive	64042388	1.00
	MatMul opt	1312786	48.78
	AVX2 + FMA	1189913	53.82
	MatMul reg	1215962	52.67
512	MatMul naive	601725497	1.00
	MatMul opt	12354883	48.70
	AVX2 + FMA	13053703	46.10
	MatMul reg	14453870	41.63
1024	MatMul naive	7557314068	1.00
	MatMul opt	93198553	81.09
	AVX2 + FMA	96108897	78.63
	MatMul reg	170681750	44.28

```
g++ -O3 -mavx2 -mfma -ffast-math
```

Анализ результатов на `-O3`

Переход к флагу `-O3` существенно меняет производительность, так как компилятор производит агрессивную автовекторизацию и более глубокий анализ графа зависимостей.

1. Произошел скачок производительности метода `MatMul opt`. На `-O2` ускорение было 7.38x, а на `-O3` оно достигло 81.09x (на размере 1024).

   На `-O3` компилятор распознает линейный паттерн доступа в методе `MatMul opt` и, по сути, самостоятельно применяет векторные инструкции `AVX2`/`FMA`.

2. Метод `MatMul reg` продолжает лидировать на средних размерах (до 128x128). На размере 128 он показал ускорение 47.26x, обгоняя даже автовекторизованный. Ручное управление регистрами (4 аккумулятора) на малых объемах данных, помещающихся в L1 кэш (32 КБ), все еще эффективнее стандартных эвристик компилятора.

3. На самой большой матрице методы `MatMul opt` (81.09x) и `AVX2` + `FMA` (78.63x) показывают практически идентичный результат, значительно обходя `MatMul reg`. При работе с данными объемом 12 МБ (это три матрицы 1024x1024), которые занимают почти весь `L3` кэш (16 МБ), важнее не скорость вычислений в ядре, а стратегия подкачки данных.

   Можно сделать вывод, что `GCC` при `-O3` не просто векторизует код, но и применяет банальный `Loop Unrolling` и, возможно, зачатки `Tiling`.

4. На `-O3` базовый метод `MatMul naive` работает почти в два раза быстрее.